

```
# OCS ONE – FAT Blocker Patches (REVISED v2)
# Validated & Corrected – 2026-08-08
# Base: Commit ebf7a3b
# Source: Independent Architecture Review Verification Report + Patch
Validation
```

Validation Summary

Blocker	Problem	Original Patch	Validator Verdict	Action
C1 – Dealer cross-contamination	Schema assumption wrong	Confirmed	Flawed	Schema migration
C2 – Dual dispatch	Scope too narrow	Confirmed	Partial	preserve GET, block writes only
H1 – Customer dealer override	Backward-compat risk	Confirmed	Partial	handle legacy products
H8 – One-shot inspection	Non-existent enums	Debatable	Flawed	keep one-shot for v1.0
H11 – Lot traceability	Balance query suspect	Confirmed	Partial	fix ledger query + add lock
H14 – Dual order completion		Confirmed	Correct	Keep as-is
H16/17/18 – Shop-floor races	7A NULL defect	Confirmed	Partial	fix 7A NULL semantics

What changed:

- Blocker 4 is **removed** from the patch list. The validator correctly identified that partial inspection introduces non-existent enum values and significant operational risk. The v1.0 resolution is operational: split large deliveries into multiple GRNs.
- Blocker 1 now includes a **required schema migration** before any route-level code can work.
- Blocker 7A fixes PostgreSQL NULL semantics in the unique index.
- Blockers 2, 3, 5 include adjustments based on validator findings.
- Blocker 6 and sub-patches 7B/7C remain unchanged (validator-approved).

```
# =====
# BLOCKER 4 – REMOVED FROM PATCH LIST
# =====
# Decision: KEEP ONE-SHOT FULL-GRN INSPECTION for v1.0
# Rationale: Partial inspection requires 4 new enum values that do not
#   exist in the schema. The current model is internally consistent.
#   Large deliveries should be split into multiple GRNs at receipt.
# Action: Document in operational runbook. Defer to v2.0.
# =====
```

```

# =====
# BLOCKER 1 - C1: Dealer Data Cross-Contamination (REVISED)
# =====
# Risk: CRITICAL
# Files: lib/db/src/schema/users.ts, middleware/auth.ts,
# routes/dealers/index.ts
# Problem: usersTable has no dealer_id column. JWT has no dealerId.
#   Any dealer-role user can read any dealer's data.
# =====

---
## STEP 1A - Schema Migration: Add dealer_id to usersTable
## File: lib/db/src/schema/users.ts
---

// BEFORE:
export const usersTable = pgTable("users", {
  id: serial("id").primaryKey(),
  email: varchar("email", { length: 255 }).notNull().unique(),
  passwordHash: varchar("password_hash", { length: 255 }).notNull(),
  name: varchar("name", { length: 255 }).notNull(),
  role: userRoleEnum("role").notNull(),
  isActive: boolean("is_active").notNull().default(true),
  createdAt: timestamp("created_at").defaultNow(),
  updatedAt: timestamp("updated_at").defaultNow(),
});

// AFTER:
export const usersTable = pgTable("users", {
  id: serial("id").primaryKey(),
  email: varchar("email", { length: 255 }).notNull().unique(),
  passwordHash: varchar("password_hash", { length: 255 }).notNull(),
  name: varchar("name", { length: 255 }).notNull(),
  role: userRoleEnum("role").notNull(),
  dealerId: uuid("dealer_id").references(() => dealersTable.id), // ADD
  isActive: boolean("is_active").notNull().default(true),
  createdAt: timestamp("created_at").defaultNow(),
  updatedAt: timestamp("updated_at").defaultNow(),
});

// Then run:
// pnpm --filter @workspace/db run generate
// pnpm --filter @workspace/db run push

---
## STEP 1B - middleware/auth.ts: Add dealerId to JWT payload type
---

// Find the AuthTokenPayload type definition (around line 9-15)
// BEFORE:
export interface AuthTokenPayload {
  userId: number;
  email: string;
  name: string;

```

```

    role: string;
  }

// AFTER:
export interface AuthTokenPayload {
  userId: number;
  email: string;
  name: string;
  role: string;
  dealerId?: string | null; // ADD
}

---
## STEP 1C – Login route: Include dealerId in JWT payload
## File: routes/auth.ts (or wherever login happens)
---

// In the login handler, after loading the user record:
const tokenPayload: AuthTokenPayload = {
  userId: user.id,
  email: user.email,
  name: user.name,
  role: user.role,
  dealerId: user.dealerId ?? null, // ADD
};

const token = jwt.sign(tokenPayload, JWT_SECRET, { expiresIn: "24h" });

---
## STEP 1D – routes/dealers/index.ts: Router-level ownership guard
---

import { requireAuth } from "@middleware/auth";

const router = express.Router();

// Apply to ALL routes under /dealers
router.use(requireAuth);

// ADD: Router-level ownership scoping
router.use((req, res, next) => {
  // Only dealer-role users are restricted to their own dealership
  if (req.user.role === "dealer") {
    const targetDealerId = req.params.id || req.params.dealerId;
    if (targetDealerId && req.user.dealerId !== targetDealerId) {
      return res.status(403).json({ error: "Access denied: not your dealership" });
    }
    // If no targetDealerId in params (e.g., list endpoint), dealer sees empty or own only
    // depending on business rules – adjust as needed
  }
  // Factory roles (director, supervisor, owner, operator, viewer) pass through
});

```

```

    next();
  });

// ... existing route definitions remain unchanged ...

---
## ACCEPTANCE CRITERIA - BLOCKER 1 (REVISED)
---

[ ] Schema migration applied: users.dealer_id column exists
[ ] Dealer user associated with DLR-001 gets 403 on GET /dealers/DLR-002/inventory
[ ] Dealer user gets 200 on GET /dealers/DLR-001/inventory
[ ] Director / owner / supervisor gets 200 on any dealer endpoint
[ ] authz certification suite passes with CERT_AUTHZ_CHECK=1
[ ] JWT payload decoded contains dealerId field for dealer users

---

# =====
# BLOCKER 2 - C2: Dual Dispatch Systems (REVISED)
# =====
# Risk: CRITICAL
# Files: routes/logistics/dispatch-orders.ts, Sidebar.tsx
# Problem: Two DIS-* numbering authorities. Legacy uses COUNT+1.
# Current write guard is requireWriteRole (allows GET for all auth).
# =====

---
## PATCH 2A - routes/logistics/dispatch-orders.ts
## Freeze legacy writes only; preserve GET for history
---

import { requireRole, requireWriteRole } from "@middleware/auth";

const router = express.Router();

// BEFORE: router.use(requireWriteRole("supervisor", "director"));
// This allowed GET for all auth users and writes for supervisor/director

// AFTER: Block ALL writes on legacy. Allow GET for any auth user
(history).
// POST / PUT / PATCH / DELETE require owner role
router.post("/", requireRole("owner"));
router.put("/:id", requireRole("owner"));
router.patch("/:id", requireRole("owner"));
router.delete("/:id", requireRole("owner"));

// GET routes remain unguarded beyond requireAuth (applied at parent
router)
// If you want to fully disable writes, return 410 Gone:
// router.post("/", (req, res) => res.status(410).json({ error: "Legacy
dispatch creation disabled. Use /dispatch" }));

```

```

// ... rest of existing route handlers ...

---
## PATCH 2B - Sidebar.tsx
## Hide legacy dispatch from non-owner roles
---

// Around line 114-122

{user.role === "owner" && (
  <SidebarLink to="/logistics/dispatch-orders">
    Legacy Dispatch (Admin Only)
  </SidebarLink>
)}

// The main "Dispatch" link (new Product Platform path) remains visible
// to all relevant roles as before.

---
## ACCEPTANCE CRITERIA - BLOCKER 2 (REVISED)
---

[ ] POST /logistics/dispatch-orders returns 403 for supervisor/director
[ ] POST /logistics/dispatch-orders returns 200/201 for owner
[ ] GET /logistics/dispatch-orders returns 200 for any authenticated user
[ ] 100-request load test on /dispatch (new path) produces zero DIS-*
collisions
[ ] Legacy dispatch link hidden from sidebar for non-owner roles
[ ] No existing reports or dashboards break due to GET access changes

---

# =====
# BLOCKER 3 - H1: Customer Dealer Override (REVISED)
# =====
# Risk: HIGH
# File: routes/customers/index.ts
# Problem: Caller-supplied dealer_id wins. Legacy-dispatched products
#   may have product.dealerId = null.
# =====

---
## PRE-REQUISITE: Audit legacy products before patch
## Run this SQL before deploying:
---

SELECT COUNT(*) FROM products
WHERE status IN ('dispatched', 'delivered_to_dealer')
AND dealer_id IS NULL;

-- If count > 0, run migration to set dealer_id from legacy dispatch
records:
-- UPDATE products p
-- SET dealer_id = ldo.dealer_id

```

```

-- FROM logistics_dispatch_orders ldo
-- WHERE p.product_serial = ldo.product_serial
--       AND p.dealer_id IS NULL
--       AND p.status IN ('dispatched', 'delivered_to_dealer');

-- After migration, verify count is 0 before applying patch.

---
## PATCH 3 - routes/customers/index.ts
## Reject caller-supplied dealer_id; derive from product with legacy
fallback
---

// BEFORE (around line 173-175):
// const dealerId = parsed.data.dealer_id ?? product.dealerId ??
"no_dealer";

// AFTER:
const { productSerial, customerName, phone, address } = req.body;

// Reject caller-supplied dealer_id
if (req.body.dealer_id !== undefined) {
  return res.status(400).json({ error: "dealer_id must not be supplied by
caller" });
}

// Look up product
const product = await db.query.products.findFirst({
  where: eq(products.productSerial, productSerial),
});
if (!product) {
  return res.status(404).json({ error: "Product not found" });
}

// Derive dealerId from product (authoritative source)
let dealerId = product.dealerId;

// Legacy fallback: if product has no dealerId, try legacy dispatch
record
if (!dealerId) {
  const legacyDispatch = await
db.query.logisticsDispatchOrders.findFirst({
  where: eq(logisticsDispatchOrders.productSerial, productSerial),
});
  if (legacyDispatch?.dealerId) {
    dealerId = legacyDispatch.dealerId;
    // Optionally: sync back to products table
    await db.update(products)
      .set({ dealerId: legacyDispatch.dealerId })
      .where(eq(products.id, product.id));
  }
}

if (!dealerId) {

```

```

    return res.status(400).json({
      error: "Product has no assigned dealer. Contact administrator for
legacy product migration."
    });
  }

```

```

// Validate dealer is active
const dealer = await db.query.dealers.findFirst({
  where: eq(dealers.id, dealerId),
});
if (!dealer || dealer.status !== "active") {
  return res.status(400).json({ error: "Product dealer is invalid or
inactive" });
}

```

```

// Use derived dealerId for registration
const registration = await db.insert(customerRegistrations).values({
  productSerial,
  dealerId,          // authoritative - never from caller
  customerName,
  phone,
  address,
  registeredAt: new Date(),
}).returning();

```

```

---
## ACCEPTANCE CRITERIA - BLOCKER 3 (REVISED)
---

```

```

[ ] POST /customers with dealer_id in body returns 400
[ ] Registration correctly inherits dealer from product.dealerId
[ ] Legacy-dispatched products with null dealerId are resolved from
logistics_dispatch_orders
[ ] Products with no dealerId and no legacy dispatch record return 400
with clear message
[ ] SELECT COUNT(*) FROM products WHERE status IN (...) AND dealer_id IS
NULL returns 0

```

```

---
# =====
# BLOCKER 5 - H11: Lot Traceability Not Enforced (REVISED)
# =====
# Risk: HIGH
# File: routes/inventory/material-issue.ts (verify exact path)
# Problem: MIN records lot references without ledger validation.
# Balance query used incorrect state filter. No concurrency lock.
# =====

```

```

---
## PRE-REQUISITE: Verify ledger schema
## Confirm these columns exist in inventory_transactions:
##   material_id, lot_number, quantity (signed), transaction_type

```

```

## If state column exists and is used for balance filtering, adjust
accordingly.
## The validator notes: signed-quantity ledger → balance is
SUM(quantity).
---

---
## PATCH 5 – routes/inventory/material-issue.ts
## Validate lot allocation against signed ledger + advisory lock
---

// ADD this validation function before the MIN creation handler:

async function validateLotAllocation(
  materialId: string,
  grnLineId: string,
  lotNumber: string,
  requestedQty: number,
  tx: any
): Promise<{ valid: boolean; error?: string }> {
  // 1. Verify GRN line exists and matches material
  const grnLine = await tx.query.grnLines.findFirst({
    where: eq(grnLines.id, grnLineId),
    with: { material: true },
  });
  if (!grnLine) {
    return { valid: false, error: `GRN line ${grnLineId} not found` };
  }
  if (grnLine.materialId !== materialId) {
    return { valid: false, error: "GRN line does not match requested
material" };
  }

  // 2. Verify lot number matches GRN line
  if (grnLine.lotNumber !== lotNumber) {
    return { valid: false, error: `Lot ${lotNumber} does not match GRN
line (expected ${grnLine.lotNumber})` };
  }

  // 3. Acquire advisory lock on (materialId, lotNumber) to prevent
concurrent over-issue
  const lockKey = `lot:${materialId}:${lotNumber}`;
  await tx.execute(sql`SELECT pg_advisory_lock(hashtext(${lockKey}))`);

  try {
    // 4. Calculate available balance from signed ledger
    // For signed-quantity ledger: balance = SUM(quantity) across ALL
transactions
    // Positive = receipt, negative = issue/consumption
    const [balanceRow] = await tx
      .select({ total: sum(inventoryTransactions.quantity) })
      .from(inventoryTransactions)
      .where(and(
        eq(inventoryTransactions.materialId, materialId),

```

```

        eq(inventoryTransactions.lotNumber, lotNumber)
    ));

    const available = Number(balanceRow?.total ?? 0);
    if (available < requestedQty) {
        return {
            valid: false,
            error: `Insufficient lot balance: available ${available},
requested ${requestedQty}`
        };
    }
    } finally {
        // Release advisory lock
        await tx.execute(sql`SELECT
pg_advisory_unlock(hashtext(${lockKey}))`);
    }

    return { valid: true };
}

// IN the MIN creation handler, REPLACE the traceability check:

// BEFORE:
// if (traceabilityRequired) {
//     // presence-only check
// }

// AFTER:
if (traceabilityRequired) {
    const lotValidation = await validateLotAllocation(
        materialId,
        grnLineId,
        lotNumber,
        requestedQty,
        tx
    );
    if (!lotValidation.valid) {
        return res.status(422).json({ error: lotValidation.error });
    }
}

// Continue with MIN creation and ledger insert...
// The ledger insert (negative quantity for issue) happens in the SAME
transaction,
// so the advisory lock ensures no concurrent MIN can over-issue.

---
## ACCEPTANCE CRITERIA – BLOCKER 5 (REVISED)
---

[ ] MIN rejected when GRN line does not match material
[ ] MIN rejected when lot number does not match GRN line
[ ] MIN rejected when lot balance < requested quantity

```

```
[ ] Concurrent MINS for same lot: exactly one succeeds if only one unit remains
[ ] Valid lot allocation succeeds and writes correct signed ledger rows
[ ] Traceability query returns validated lot references only
```

```
# =====
# BLOCKER 6 – H14: Dual Order Completion Paths
# =====
# Risk: HIGH
# Files: routes/manufacturing/product-creation.ts,
#        routes/manufacturing/stages.ts,
#        routes/manufacturing/qc-approval.ts
# Status:  VALIDATOR FULLY APPROVED – no changes needed
# =====
```

```
## PATCH 6A – routes/manufacturing/product-creation.ts
## Add idempotency guard (unchanged from v1 – approved)
```

```
export async function completeOrderWithProduct(
  orderId: string,
  tx: any
): Promise<void> {
  // ===== IDEMPOTENCY GUARD =====
  const [order] = await tx
    .select()
    .from(mfgProductionOrders)
    .where(eq(mfgProductionOrders.id, orderId))
    .for("update");

  if (!order) {
    throw new Error(`Production order ${orderId} not found`);
  }

  if (order.status === "completed") {
    console.warn(`completeOrderWithProduct: order ${orderId} already
completed – skipping`);
    return;
  }

  if (order.status !== "qc_approved" && order.status !==
"final_stage_completed") {
    throw new Error(`Cannot complete order ${orderId} from status:
${order.status}`);
  }
  // ===== END IDEMPOTENCY GUARD =====

  // ... existing completion logic ...

  await tx
    .update(mfgProductionOrders)
```

```

        .set({ status: "completed", completedAt: new Date() })
        .where(eq(mfgProductionOrders.id, orderId));
    }

---
## PATCH 6B - routes/manufacturing/stages.ts
## Guard terminal stage completion (unchanged from v1 - approved)
---

// Around line 587-592
if (order.status !== "completed") {
    await completeOrderWithProduct(orderId, tx);
} else {
    console.warn(`Stage completion: order ${orderId} already completed`);
}

---
## PATCH 6C - routes/manufacturing/qc-approval.ts
## Guard QC approval completion (unchanged from v1 - approved)
---

// Around line 126-130
if (order.status !== "completed") {
    await completeOrderWithProduct(orderId, tx);
} else {
    console.warn(`QC approval: order ${orderId} already completed`);
}

---
## ACCEPTANCE CRITERIA - BLOCKER 6
---

[ ] Concurrent terminal stage completion + QC approval does not create
duplicate products
[ ] Second call to completeOrderWithProduct on same order is a no-op
[ ] Order status transitions to "completed" exactly once
[ ] Only one product record created per production order
[ ] Audit trail shows single completion event

---

# =====
# BLOCKER 7 - H16+H17+H18: Shop-Floor Race Conditions (REVISED)
# =====
# Risk: HIGH
# Files: lib/db/src/schema.ts, routes/manufacturing/stages.ts,
#         routes/manufacturing/matches.ts
# Problem: 7A had PostgreSQL NULL semantics defect. 7B and 7C approved.
# =====

---
## PATCH 7A - lib/db/src/schema.ts (REVISED)
## Fix genealogy uniqueness constraint for PostgreSQL NULL semantics
---
```

```
// PostgreSQL treats NULLs as not equal in unique indexes.
// If componentId or serialNumber can be NULL, duplicates are allowed.
// Fix: Use COALESCE with sentinel values in the unique index expression.
```

```
// BEFORE (v1 - flawed):
// uniqueIndex("unique_genealogy").on(
//   table.productionOrderId,
//   table.componentType,
//   table.componentId,
//   table.serialNumber
// )
```

```
// AFTER (v2 - corrected):
export const productGenealogy = pgTable("product_genealogy", {
  id: serial("id").primaryKey(),
  productionOrderId: integer("production_order_id").notNull(),
  componentType: varchar("component_type", { length: 50 }).notNull(),
  componentId: integer("component_id"),
  serialNumber: varchar("serial_number", { length: 100 }),
  // ... other columns
}, (table) => ({
  uniqueGenealogy: uniqueIndex("unique_genealogy").on(
    table.productionOrderId,
    table.componentType,
    sql`COALESCE(${table.componentId}, 0)`,
    sql`COALESCE(${table.serialNumber}, '')`
  ),
}));
```

```
// Alternative if you prefer NOT NULL with sentinels:
// componentId: integer("component_id").notNull().default(0),
// serialNumber: varchar("serial_number", { length: 100
// }).notNull().default(''),
// Then use plain .on(table.productionOrderId, table.componentType,
// table.componentId, table.serialNumber)
```

```
---
## PATCH 7B - routes/manufacturing/stages.ts (UNCHANGED - approved)
## Charger reservation/release with conditional predicates
---
```

```
// --- RESERVATION (around line 68-83) ---
const [reserved] = await tx
  .update(chargers)
  .set({ status: "in_use", productionOrderId: orderId, reservedAt: new
Date() })
  .where(and(
    eq(chargers.id, chargerId),
    eq(chargers.status, "available")
  ))
  .returning();
```

```
if (!reserved) {
```

```

    throw new Error(`Charger ${chargerId} is no longer available – possibly
reserved by another operator`);
}

// --- RELEASE (around line 170-177) ---
const [released] = await tx
    .update(chargers)
    .set({ status: "available", productionOrderId: null, reservedAt: null
})
    .where(and(
        eq(chargers.id, chargerId),
        eq(chargers.status, "in_use"),
        eq(chargers.productionOrderId, orderId)
    ))
    .returning();

if (!released) {
    throw new Error(`Charger ${chargerId} release failed – may have been
released already or reassigned`);
}

---
## PATCH 7C – routes/manufacturing/matches.ts (UNCHANGED – approved)
## Atomic cell matching with transaction-level locking
---

const { orderId, cellIds } = req.body;

await db.transaction(async (tx) => {
    // 1. Lock and verify cells are still approved INSIDE the transaction
    const lockedCells = await tx
        .select()
        .from(cellRecords)
        .where(and(
            inArray(cellRecords.id, cellIds),
            eq(cellRecords.status, "approved")
        ))
        .for("update");

    if (lockedCells.length !== cellIds.length) {
        const foundIds = new Set(lockedCells.map(c => c.id));
        const missing = cellIds.filter(id => !foundIds.has(id));
        throw new Error(`Cells no longer available: ${missing.join(", ")}`);
    }

    // 2. Allocate cells atomically
    await tx
        .update(cellRecords)
        .set({ status: "allocated", productionOrderId: orderId, allocatedAt:
new Date() })
        .where(and(inArray(cellRecords.id, cellIds), eq(cellRecords.status,
"approved")));

```

```

    // 3. Create genealogy records (idempotent - duplicates ignored due to
7A unique constraint)
    for (const cell of lockedCells) {
        await tx
            .insert(productGenealogy)
            .values({
                productionOrderId: orderId,
                componentType: "cell",
                componentId: cell.id,
                serialNumber: cell.serialNumber,
            })
            .onConflictDoNothing({
                target: [productGenealogy.productionOrderId,
productGenealogy.componentType, productGenealogy.componentId]
            });
    }
});

```

```

---
## ACCEPTANCE CRITERIA - BLOCKER 7 (REVISED)
---

```

```

[ ] Two concurrent charger reservations for same charger - only one
succeeds
[ ] Two concurrent cell matching requests for same cells - only one
succeeds
[ ] Genealogy insert with duplicate key is silently ignored (idempotent)
[ ] Genealogy unique constraint prevents duplicates even when componentId
or serialNumber is NULL
[ ] Load test with 10 concurrent operators - zero double-allocations
[ ] Charger release by wrong order fails
[ ] Cell allocation rollback on partial failure (all-or-nothing)

```

```

---
# =====
# REVISED POST-PATCH CHECKLIST
# =====

```

```

## Schema Migration Order (REVISED)
1. STEP 1A: Add dealer_id to usersTable
2. PATCH 7A: Fix genealogy unique constraint (COALESCE)
3. Generate and apply Drizzle migrations
4. Verify constraints in database

```

```

## Code Deployment Order (REVISED)
1. STEP 1B + 1C: AuthTokenPayload type + login route dealerId
2. STEP 1D: Router-level dealer ownership guard
3. PATCH 2A + 2B: Dispatch write freeze (preserve GET)
4. PRE-REQ 3: Audit + migrate legacy products with null dealerId
5. PATCH 3: Customer dealer-override (with legacy fallback)
6. PATCH 5: Lot traceability validation (advisory lock + signed ledger)
7. PATCH 6A + 6B + 6C: Order completion idempotency
8. PATCH 7B + 7C: Shop-floor race conditions

```

```
## Blocker 4 – Operational Resolution (NO CODE CHANGE)
- [ ] Update operational runbook: "Large deliveries → split into multiple GRNs"
- [ ] Train receiving supervisors on multi-GRN workflow
- [ ] Defer partial inspection to v2.0 roadmap
```

Regression Tests

```
[ ] authz certification suite – authoritative mode
[ ] audit certification suite – authoritative mode
[ ] config certification suite
[ ] End-to-end traceability: GRN → cell → battery → dispatch → customer → warranty
[ ] Dealer data isolation: supervisor cannot see cross-dealer data
[ ] Concurrent cell matching: 10 parallel requests, 0 duplicates
[ ] Concurrent charger reservation: 10 parallel requests, 0 double-bookings
[ ] Concurrent MIN issuance: 10 parallel requests for same lot, no over-issue
[ ] Load test: 100 DIS-* numbers on new path, 0 collisions
[ ] Legacy dispatch GET history still accessible to supervisors/directors
```

FAT Freeze Criteria (REVISED)

```
[ ] All 6 code blockers closed and regression-tested (Blocker 4 resolved operationally)
[ ] Schema migrations applied and verified
[ ] Legacy product dealerId audit returns 0 nulls
[ ] No legacy dispatch writes during test (GET allowed for history)
[ ] Cell matching tested with two concurrent users – no duplicate allocations
[ ] Charger reservation tested with concurrent stage starts – no double-booking
[ ] MIN lot validation tested with concurrent requests – no over-issue
[ ] End-to-end traceability demonstrated in one transaction sequence
[ ] Dealer portal isolation verified
```

```
# End of Revised Patches
```